Now let's update our chat page component (in page.js) to utilize these new hooks and actions. We'll replace the previous state management with our newly integrated state system.

Listing 4.9 Including new hooks for managing AI and UI state

```
export default function Home() {
  const [provider, setProvider] = useState('openai');
  const [model, setModel] = useState('gpt-4');
  const [input, setInput] = useState('');
  const [isLoading, setIsloading] = useState(false);
  const [conversation, setConversation] =
useUIState();
  const [aiState] = useAIState();
  const { continueConversation } = useActions();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setUIState((prev) => ({.
      ...prev,
      messages: [...prev.messages, { role: 'user', content: input }],
    }));
    const response =
await continueConversation(input, provider, model);
    setUIState((prev) => ({
      ...prev,
      messages: [...prev.messages, response],
    }));

    setInput('');
  };
```

Uses useUIState to manage UI-specific state

Uses useAIState to access the AI state (read-only)

Retrieves the continueConversation action from the AI context

Calls the setUIState to update the UI state here

Calls the server action to process user input and get AI response

We've replaced the `useState` for conversation management with `useUIState`. This hook gives us access to the UI state defined in our AI context. We've also added `useAIState` to access the AI state. While we're not using it directly in this example, it's available for more complex scenarios where we might need to reference the AI's understanding of the conversation. The `continueConversation` function is now retrieved using the `useActions` hook, ensuring we're using the server action defined in our AI context.

With the modifications we've made, our application now uses a more robust and scalable state management system. This upgrade significantly enhances our ability to manage complex state in AI-powered interfaces without introducing visual changes to the user experience.

As an improvement, notice that we keep both the `isLoading` and `value` state variables outside of the UI state. See if you can include this inside the UI state instead. What changes do you think you need to make this work? I will leave this to you as a small exercise.

TESTING THE CHANGES

After implementing these modifications, try to rebuild your application and visit the main chat interface. Pay special attention to loading indicators and disabled states